URMACK · FLAGSHIP PRODUCT · CONFIDENTIAL

Urmack Watch

The build guide: which chains we monitor and in what order, the exact technology stack, the cost model, and the engineering nitty-gritty required to ship a multi-tenant DeFi invariant-monitoring SaaS.

Document 04 — Tech Stack & Chain Strategy
Companion documents 01 System Architecture · 02 Flows & SDLC Roadmap · 03 Product Designs
Version 1.0 · 16 October 2026
Author Office of the Interim CTO, Urmack
Status Approved for build — governs all Phase 0–2 engineering decisions

1. The decision at a glance

DECISION	CHOICE	WHY (SHORT FORM)
Chain strategy	Ethereum L1 first, then Base + Arbitrum One (Phase 1–2); OP Stack family next; Solana only in Phase 3 behind an adapter interface	Ethereum holds ~$49.7B of ~$65B+ tracked DeFi TVL; L2s add cheap, fast blocks where the same EVM adapter works unchanged
Language / runtime	TypeScript (strict) everywhere, Node 22 LTS, `viem` for chain I/O	One language across portal, API, compiler and workers; viem's typed ABIs make spec compilation a type-level problem
Architecture shape	pnpm monorepo — Next.js portal · Hono API · scheduler · evaluator workers	Control plane and evaluation plane deploy independently (Doc 01 §3)
Data acquisition	**Direct RPC polling is the product.** Indexers (Ponder/Goldsky) are optional for evidence and dashboards, never on the critical path	Invariants read *current state* via batched `eth_call`; a third-party indexer adds a dependency we'd have to trust for our own alerts
Storage	Supabase Postgres (RLS = tenancy) + TimescaleDB hypertable for evaluations + S3/R2 evidence objects	Already in the stack; RLS gives hard multi-tenant isolation with almost no code
Jobs / coordination	BullMQ on Redis, block-height-driven ticks, one-in-flight lock per plan	Boring, proven, retries and backpressure for free
Alert delivery	HMAC-signed webhooks + Telegram / Discord / Slack / PagerDuty / email; WhatsApp Cloud API on premium	Meets buyers where their on-call already lives
Billing	Stripe Billing (metered overages on evaluations + chains)	0.7% PAYG; metered usage without re-platforming later
Hosting	Portal + API on Vercel/Fly; workers on Fly.io or Render (long-running, region-pinned near RPC); Cloudflare only for edge/WAF	Workers need sticky state (Redis, sockets) and predictable egress — a plain container beats serverless here

2. Which blockchain? Chain selection and rollout order

Anchor: Ethereum. Not because it is fashionable, but because that is where the collateral, the stablecoins and the buyers are. TVL and protocol counts below are DeFiLlama snapshots of **16 October 2026**.

#	CHAIN	DEFI TVL	PROTOCOLS	WHY IT MATTERS TO URMACK WATCH
1	**Ethereum L1**	$49.7B	1,680	Anchor deployment. Lending (Aave, Morpho, Spark), LST/LRT restaking (Lido, EigenLayer, ether.fi), stablecoins — exactly the invariant families we sell (solvency, share price, collateral ratio).
2	**Base**	$6.13B	1,018	Second by TVL; large Morpho/Aave footprint, ~$5.26B stablecoins. Same EVM + OP Stack finality model as #7, so marginal integration cost is near zero.
3	**Solana**	$5.91B	551	Large and active (2.9M daily active addresses, $2.35B daily perps) but a different runtime — no `eth_call`, no shared ABI path. Phase 3 behind the adapter interface, not before.
4	**BNB Chain**	$5.47B	1,140	Retail DeFi scale (PancakeSwap, Venus), cheap EVM. Low priority for security-budget buyers; Phase 2.
5	**Arbitrum One**	$1.40B	1,014	**Priority above its TVL rank:** $3.39B daily perps volume (GMX et al.) — perps/cross-margin invariants are high-value and underserved.
6	**Polygon PoS**	$676M	776	Aave deployments + prediction markets. Phase 2.
7	**Optimism**	$443M	429	Smaller footprint, but OP Stack — adds as config once Base works.
8	**Avalanche**	$596M	570	Lending + RWA niche; on demand.
—	*Watchlist*	Hyperliquid $1.04B TVL / $13.5B daily perps · Monad $992M		Hyperliquid matters if perps risk is in scope but needs its own integration assessment; Monad is young. Re-evaluate quarterly.

> **Chain decision.** Phase 1 ships **Ethereum + Base + Arbitrum One** on one EVM adapter: it covers ~$57B of TVL and every invariant category we sell, and Arbitrum supplies the perps exposure. Phase 2 adds the remaining OP Stack chains, Polygon and BNB as *configuration*, not new code. Solana is a Phase 3 adapter, gated on demand from a design partner. **We are chain-agnostic in the spec language and chain-specific in the adapter** — a spec says "contract X on chain Y", never "call this RPC".

2.1 Finality policy — the nitty-gritty that decides alert credibility

Block-count depth rules are wrong on L2s. The meaningful states are: *sequencer-ordered → posted to L1 → L1-finalized*. We model finality as a **state** attached to every evaluation, and alert latency is decoupled from confidence:

CHAIN	BLOCK CADENCE	INTERMEDIATE ("SAFE")	STRONG FINALITY	URMACK WATCH POLICY
Ethereum	12 s slots; epoch = 6.4 min	RPC `safe` tag	`finalized` ≈ 12.8 min (~64 blocks)	Detect on `latest`, fire Critical **immediately**, tag the alert `provisional`; auto-escalate to `finalized` ~13 min later or **retract loudly** on reorg
Arbitrum One	250 ms ordering	Batch posted to L1	~15–19 min after L1 batch inclusion	Detect on sequencer feed (`provisional`); confirm when the batch finalizes on L1
Base	Flashblock ~200 ms; L2 block 2 s	L1 batch ≈ 2 min	≈ 20 min	Detect on Flashblock head; confirm on L1 batch finality
Optimism	2 s blocks	`safe` = data posted to L1	15–30 min after submission	Same as Base (shared OP Stack semantics)

> **Two rules.** (1) *Alert fast, retract loudly* — a Critical invariant breach waits for nothing; correctness of the retraction is what earns trust. (2) The 7-day Standard Bridge withdrawal delay is **not** transaction finality, and neither is a sequencer's sub-second soft confirmation. Doc 01 §7's heartbeat covers the case where our own view of the head stops moving.

3. The stack, layer by layer

1 · Portal `NEXT.JS 15 · TS STRICT · TAILWIND`

App Router, server components for dashboards, client islands for the invariant editor (YAML editor + live dry-run results). Auth via Supabase magic link in Phase 1, SSO/SAML for enterprise in Phase 3. Deployment: Vercel (or Fly for EU residency).

2 · API `HONO`

Fetch-standard typed routes: orgs, protocols, contracts, specs, routes, alert query. Runs on Node 22 (Fly) — same binary type as workers, one Dockerfile. Zod-validated bodies, OpenAPI generated from routes. `GET` responses carrying tenant data send `Cache-Control: no-store`.

3 · Spec compiler `YAML → PLAN`

`packages/spec-lang`: YAML spec → JSON Schema → Zod-validated **evaluation plan** (a `viem` multicall payload + assertion function + finality policy + thresholds). Plan versions are immutable and checksummed; the evaluator never reads YAML at runtime.

4 · Scheduler `NODE · REDIS`

Subscribes to block headers per chain (`viem` `watchBlockNumber` with polling fallback). On tick: claim each due plan via `SET NX PX` lock, enqueue on BullMQ. Emits heartbeat if `now - last_eval > 3 × interval`.

5 · Evaluator workers `BULLMQ · VIEM`

Pull job → build consistent snapshot at one block number (batched `eth_call` multicall3 + bounded `getLogs` window) → run assertion (point-in-time / temporal / derived) → write result (pass *and* margin) → if breach, hold until finality state allows and hand to router. Horizontal scale = more workers; one plan in flight at a time.

6 · Alert router `TYPESCRIPT`

Severity classification → dedup key (`spec_id + breach_kind`) → cooldown → evidence pack assembly → fan-out adapters (webhook HMAC-SHA256 over `ts.body` with 5-min replay window; Telegram/Discord/Slack bots; PagerDuty Events API v2; Resend email; WhatsApp Cloud API on premium). Retries: 5× exponential with jitter; dead-letter to the alert record.

7 · Storage `SUPABASE · TIMESCALE · R2`

Postgres + RLS for tenancy/specs/alerts/audit; Timescale hypertable `evaluation` partitioned by plan, retention policy 90 d raw / 2 y hourly; Upstash/Redis for locks and circuit breakers; Cloudflare R2 (or S3) for immutable evidence packs with 15-minute signed URLs.

3.1 Monorepo layout

```
# pnpm workspace + turborepo — one repo, one CI pipeline
urmack-watch/
├─ apps/
│  ├─ portal/            # Next.js 15 client portal
│  ├─ api/               # Hono REST API (control plane)
│  ├─ scheduler/         # block watchers, tick emission, heartbeat
│  ├─ evaluator/         # BullMQ workers — the evaluation plane
│  └─ router/            # severity, dedup, evidence, channel adapters
├─ packages/
│  ├─ spec-lang/         # YAML DSL, JSON Schema, plan compiler (Zod)
│  ├─ chain/             # viem clients, per-chain adapters, finality policy
│  ├─ severity/          # classification + cooldown rules
│  ├─ evidence/          # evidence-pack builder, S3 writer
│  └─ db/                # migrations, RLS policies, typed client
├─ infra/
│  ├─ docker-compose.yml # postgres+timescale, redis, minio, mailpit
│  └─ terraform/         # fly.io apps, R2, DNS, secrets
└─ .github/workflows/    # ci.yml: lint → typecheck → unit → e2e → deploy
```

3.2 Local development & environments

- **Anvil (Foundry)** fork of Ethereum mainnet for integration tests — a fixture protocol with a deliberately breakable invariant (drain the reserve, skew the share price) proves the whole pipeline end-to-end in CI.
- Three environments: `local` (docker-compose + Anvil) · `staging` (real RPC, synthetic protocol, no real routes) · `prod`. Spec fixtures and alert-route fixtures are shared via seed scripts.
- CI gates: `tsc --noEmit`, ESLint, Vitest unit + Anvil integration, RLS policy tests (two tenants, cross-tenant read must 404/403), and a 24-hour shadow run of the scheduler before any production deploy touching the evaluation plane.

4. Data acquisition: RPC-first, indexers optional

The product's critical path reads *current state*: balances, ratios, share prices, privileged-role changes. That is `eth_call` + `getLogs` work, not a graph query. Building on an indexer would mean our alerts depend on a third party's sync lag — unacceptable for a product whose promise is "we tell you the moment an invariant breaks".

OPTION	SHAPE	ROLE FOR URMACK WATCH
Alchemy / QuickNode	Managed RPC; Alchemy free 30M CU/mo then $0.525/M CU; QuickNode Build 80M credits/$49 mo ($34 annual), overage $0.62/M	**Primary.** Two providers simultaneously from day one (different vendors, not two keys of one) for circuit-breaker failover.
dRPC	PAYG, 210M free CU/mo, ~$6/M requests	Cheap overflow tier for backfills and historical evidence gathering.
Self-hosted Erigon + Lighthouse	Your hardware, no meter — plus sync, disk, bandwidth, on-call	Phase 3 option once one chain's RPC bill > ~$3k/mo, or for archive-heavy evidence queries. Not before.
Ponder / SQD / Goldsky / The Graph	Self-hosted TS indexer; managed pipelines ($5/M RPC on SQD/Goldsky, subgraph worker time $0.05/h, Graph Studio 100k free queries then $2/100k)	**Non-critical only:** portfolio dashboards, "30-day margin history" precomputation, deep incident forensics. If it lags, we lose a chart — not an alert.

> **Rule: nothing on the alert path may have a sync lag we do not control.** The indexer earns its place only after Phase 1, and only where it demonstrably removes RPC load.

4.1 RPC cost model (per protocol per month)

Assumption: one snapshot ≈ 150 CU (one multicall ≈ 26 CU + bounded `getLogs` + a few reads). Real numbers come from Phase 0's spike — this is the budget to design against:

CADENCE	TICKS / DAY	CU / MONTH	AT $0.525/M CU	NOTES
5 min (Tier 1)	288	1.3M	**≈ $0.70**	Baseline plan; RPC cost is noise
1 min	1,440	6.5M	≈ $3.40	Most Tier 2 protocols
10 s	8,640	39M	≈ $20.50	Perps / high-value lending
Per block (L2 @ 2 s)	43,200	194M	≈ $102	Only where a customer pays for it

Multiply by 2 for historical backfill, traces and evidence re-queries. Even a 20-protocol Tier-2 portfolio lands under ~$1k/mo in RPC — **the economics of this product are not gated by compute; they are gated by distribution.**

4.2 Monthly infrastructure budget

LINE ITEM	PHASE 1 (ALPHA)	PHASE 2-3 (BETA → GA)
Supabase Pro (+ disk/Timescale)	$25–50	$100–250 (compute add-ons, PITR)
Redis (Upstash / managed)	$0–20	$50–150
Workers + API compute (Fly/Render)	$50–100	$300–800 (evaluator scales with protocols)
RPC providers (two vendors)	$0–50	$300–1,500
R2 evidence storage + egress	< $10	$20–100
Alert/email providers, domain, WAF	$30–60	$100–300
Observability (OTel → Grafana Cloud, Sentry)	$0–50	$150–400
Total	**≈ $160–340 / mo**	**≈ $1,000–3,500 / mo**

One design-partner contract at the recommended entry price covers Phase 1 infrastructure with room to spare (see §8).

5. The spec language and evaluation loop (the actual product)

Everything above is plumbing. The asset is the language and the engine:

```
# invariant: eth-solvency-usdc — declarative, human-auditable
name: Reserve solvency (USDC vault)
protocol: acme-vault
kind: point_in_time            # point_in_time | temporal | derived
cadence: 60s
severity: critical
observe:
  assets:   { call: "totalAssets()" }
  liabilities: { call: "totalLiabilities()" }
assert: assets ⩾ liabilities * 1.02   # 2% safety band → Warning at 1.05
temporal: holds_for: 3_blocks          # suppresses single-block noise
finalize: alert_at: provisional        # confirm_at: finalized (§2.1)
```

1. **Compile.** Resolves contract addresses + proxies, expands `observe` into a multicall3 payload, type-checks return types against the ABI, emits `EvalPlan{version, checksum}`.
2. **Dry-run gate.** The plan replays over 7 days of history; a spec that fails or throws cannot be armed. This is the false-positive firewall.
3. **Arm.** Scheduler picks it up at its cadence; every evaluation — pass or fail — writes `(plan, block, ts, values, pass, margin)`.
4. **Breach.** Hold to finality policy → classify → dedup/cooldown → evidence pack → route. Margin history powers the "how close did we come?" chart, which is the feature that keeps customers after the adrenaline fades.

6. Security & multi-tenancy nitty-gritty

CONCERN	IMPLEMENTATION
Signing material	**None. Ever.** The product holds read-only RPC credentials only; no wallet, no signer, no key-management service is provisioned in any environment.
Tenant isolation	Postgres RLS on every table with `org_id`; the API's DB role is *not* `service_role` in request context; cross-tenant probes are a CI test, and an RLS pen-test is a Phase 2 exit gate.
Secrets	GitHub OIDC → cloud secret manager (no long-lived CI keys); `age/SOPS`-encrypted repo config for local dev; provider keys scoped per chain and rotated quarterly; webhook HMAC keys per route.
Supply chain	pnpm lockfile + `pnpm audit` in CI, Dependabot/Renovate, SBOM per release, no postinstall scripts from new deps without review.
Auditability	Append-only `audit_log` (who changed a spec/route/severity, with before/after); alert status transitions (open → acked → resolved / false-positive) are part of the audit trail.

CONCERN	IMPLEMENTATION
Evidence objects	Immutable, 90-day retention, 15-min signed URLs, served no-store; exportable as a bundle for IR retainers.
Abuse resistance	Per-org rate limits at the API, per-route alert rate caps, re-auth (step-up) for routing/severity changes.

7. Integration & delivery nitty-gritty

Outbound (alert channels)

- **Webhook:** `POST` JSON, headers `X-Urmack-Signature: sha256=…` , `X-Urmack-Timestamp` (±5 min), `X-Urmack-Alert-Id` as idempotency key; verify snippet published in docs for JS/Python/Go.
- **Retries:** 5 attempts, exponential backoff + jitter, then DLQ → visible as `delivery_failed` on the alert, not silent.
- **PagerDuty:** Events API v2, severity mapping Critical→critical, Warning→warning.
- **Telegram/Discord/Slack:** bot per workspace; message contains severity, invariant, observed vs. threshold, block, link to evidence.
- **WhatsApp Cloud API:** premium tier only, template messages, opt-in captured in portal.
- **Inbound:** Slack interactivity (ack/resolve from the channel) writes back through the API with signed verification.

Control & operations

- **Auth:** Supabase Auth (magic link → passkeys → enterprise SSO). Session in HTTP-only cookie; no tokens in localStorage.
- **Billing:** Stripe — flat platform fee + metered overage (evaluations beyond plan cadence, chains beyond plan count); webhook → `billing_status` with grace-period degradation (degrade cadence, never delete data).
- **Observability:** OpenTelemetry traces across scheduler → worker → router; RED metrics per plan; Grafana Cloud alert on *our own* eval SLO (99.9% of expected ticks completed) plus dead-man on heartbeat.
- **Feature flags / config:** env-driven per-environment config, no hard-coded chain lists — chains are rows in a `chain_config` table.
- **Backups:** PITR on Postgres, cross-region R2 replication for evidence, quarterly restore drill recorded in the runbook.

8. Packaging & price sanity-check

Competitive reference points (Oct 2026): **Forta** sells token-denominated subscriptions (General Plan announced at 250 FORT/mo; Premium Feeds priced per feed) — no transparent USD anchor. **Hypernative** is demo/quote-led with no public price. **Tenderly** alerts are paywalled, sales-gated. **Chainlink Automation was deprecated in mid-2026** (CRE is the successor), removing the one gas-plus-premium incumbent reference. *No competitor publishes a clean "economic invariant monitoring" price — which is the opening.*

TIER	PRICE	INCLUDED	OVERAGE
Sentinel	$499 / mo	1 protocol, 3 chains (Phase 2), 10 invariants, 5-min cadence, 90-day history, webhook/Telegram/Discord/email	$150 / extra chain

TIER	PRICE	INCLUDED	OVERAGE
Guardian	$1,999 / mo	3 protocols, 1-min cadence (10 s on request), unlimited invariants, PagerDuty + Slack, 1-year history, evidence packs, 30-min response SLA	$400 / extra protocol
Sentinel+ IR	Custom (≥ $4k / mo)	Portfolio coverage, per-block cadence, private invariants, IR retainer hand-off, dedicated Slack channel, custom chains	Contractual

Rationale: at these prices the whole infrastructure bill (§4.2) is a rounding error against a single mid-tier customer, and we price against *assurance* — invariant coverage, latency, evidence, response — not against raw alerts, where Forta/Hypernative would commoditize us. Revisit after Phase 1 design-partner feedback.

9. 90-day build order (what to write, in sequence)

WEEK	DELIVERABLE	EXIT CRITERION
1–2 (Phase 0)	Spikes: RPC multi-provider failover; `spec-lang` compiler for `point_in_time`; Anvil fixture protocol	One invariant evaluated against Anvil + one against Ethereum mainnet read-only, both producing `EvaluationResult`
3–4	Scheduler + evaluator + Timescale write path + heartbeat dead-man	24 h unattended run, zero missed ticks, synthetic breach → alert in < 60 s
5–6	Alert router + evidence pack v1 + webhook/Telegram	Signed webhook verified by a reference consumer; evidence bundle opens standalone
7–8	Supabase RLS tenancy + Hono API + portal read views	Two-tenant isolation test green; design partner can view their first live invariant
9–10	Portal editor + dry-run gate + Base & Arbitrum adapters	Self-serve spec authoring → 7-day dry-run → armed, without engineering staff
11–12	Stripe billing, onboarding flow, staging hardening, docs	Phase 1 gate: 30 consecutive days of correct evaluation begins (Doc 01 §2)

10. Sources

DeFiLlama chain TVL pages (16 Oct 2026 snapshots) — defillama.com/chain/ethereum · /chain/base · /chain/solana · /chain/bsc · /chain/arbitrum · /chain/polygon · /chain/avalanche · /chain/op-mainnet · /chain/hyperliquid-l1 · /chain/monad
Finality: ethereum.org/developers/docs/consensus-mechanisms/pos/ · docs.base.org/specifications/transactions/transaction-finality · docs.optimism.io/op-stack/transactions/transaction-finality · docs.arbitrum.io/how-arbitrum-works/deep-dives/sequencer · docs.arbitrum.io/how-arbitrum-works/inside-arbitrum-nitro · developers.circle.com/cctp/concepts/finality-and-block-confirmations
Protocol/upgrade context: blog.ethereum.org/2025/04/23/pectra-mainnet · blog.ethereum.org/2025/11/06/fusaka-mainnet-announcement · docs.arbitrum.io/how-arbitrum-works/bold/gentle-introduction · l2beat.com/layer2s/summary
Pricing: alchemy.com/pricing · quicknode.com/pricing · drpc.org/pricing · supabase.com/pricing · neon.com/pricing · stripe.com/billing/pricing · docs.sqd.dev/en/cloud/pricing/overview · goldsky.com/pricing · thegraph.com/studio-pricing · docs.forta.network/en/latest/fees-faqs/ · hypernative.io/product/onchain-monitoring-automated-response ·

docs.tenderly.co/monitoring/alerts/introduction · docs.chain.link/cre/reference/cla-migration-ts
Indexer self-hosting: ponder.sh/docs/production/self-hosting · docs.sqd.dev/en/sdk/self-hosting · github.com/erigontech/erigon

---

Urmack Watch — Tech Stack & Chain Strategy · v1.0 · 16 Oct 2026 · Companion: 01 System Architecture, 02 Flows & SDLC Roadmap, 03 Product Designs